Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Laboratory Work 1:

Laboratory Work 1: Concurrency in Practice

Elaborated:
Ilie Covali

gr. FAF-243

Verified:

asist. univ. Cristofor Fistic

Chișinău – 2026

Contents

1 Objective and Requirements **2**
1.1 Laboratory Objective 2
1.2 Hardware and Runtime Environment 2
1.3 Dataset Generation 2

2 Task 1a: Direct File Partitioning and Multithreaded Sign Counting **3**
2.1 Methodology and Boundary Handling 3
2.2 Experimental Results 3
2.3 Discussion and Analysis 4

3 Task 1b: Three Synchronization Strategies for Sign Counting **5**
3.1 Experimental Setup 5
3.2 Experimental Results 5
3.3 Discussion and Analysis 6

4 Task 1c: Producer and Consumers Pipeline **7**

5 Task 2: Many Files on Slow Storage **7**

6 Control Questions and Conclusion **7**

1 Objective and Requirements

1.1 Laboratory Objective

The objective of this laboratory work is to investigate real-world multithreaded concurrency in systems programming, measuring how thread counts and workload partitioning strategies influence execution time, parallel speed-up, and peak memory utilization. Key goals include:

1. Observing CPU-bound workload scaling across physical cores and logical processors.
2. Identifying parallel speed-up saturation points and context-switching penalties.
3. Demonstrating race conditions and non-deterministic memory corruption in unsynchronized concurrent updates.
4. Quantifying the performance overhead of mutual exclusion (locks) compared to thread-local aggregation.

1.2 Hardware and Runtime Environment

To ensure true OS-level thread concurrency without the Global Interpreter Lock (GIL), the implementation uses the newly released **free-threaded CPython 3.13** runtime, managed via the `uv` package and virtual environment manager.

Table 1: Hardware and Environment Specifications

Component	**Specification**
CPU Model	AMD Ryzen 3 4300U with Radeon Graphics
Architecture	x86_64 (Zen 2)
Physical Cores	4
Logical Processors	4 (1 thread per core, SMT disabled)
RAM	19.4 GB DDR4
Storage Subsystem	WDC PC SN530 NVMe SSD (256 GB)
Operating System	CachyOS Linux (Kernel 7.2.9-2-cachyos)
Programming Language	Python 3.13.16+freethreaded (Clang 23.1.3 build)
GIL Status	Disabled (`sys._is_gil_enabled() == False`)
Environment Tool	`uv` 0.12+

1.3 Dataset Generation

The input dataset consists of a text file `numbers.txt` containing exactly 50 000 000 uniformly distributed pseudo-random integers in the range $[-1\,000\,000, 1\,000\,000]$, formatted with one number per line. The file size is approximately 352.33 MB.

$$\text{Total} = \text{Count}(\text{negative}) + \text{Count}(\text{positive}) + \text{Count}(\text{zero}) = 50\,000\,000 \tag{1}$$

Every implemented task must satisfy this strict conservation invariant.

2 Task 1a: Direct File Partitioning and Multithreaded Sign Counting

2.1 Methodology and Boundary Handling

Task 1a evaluates parallel file reading and counting across thread configurations $T \in \{1,2,4,8,16,32,64\}$. Reading the entire file on a single thread or pre-loading the text into RAM is prohibited. The file of size S bytes is partitioned into T disjoint nominal byte intervals:

$$\text{Range}_t = \left[\left\lfloor \frac{t \cdot S}{T} \right\rfloor, \left\lfloor \frac{(t+1) \cdot S}{T} \right\rfloor \right), \quad t \in \{0,1,\ldots,T-1\} \tag{2}$$

Because nominal byte offsets arbitrarily split text lines, a boundary reconciliation algorithm is implemented:

- **Initial Seek:** Thread $t > 0$ seeks to byte offset $\text{start}_t - 1$ and inspects the preceding character. If the preceding character is a newline (`\n`), the thread begins parsing immediately at start_t. If it is not a newline, the thread discards the remainder of the fractional line via `readline()` and begins parsing from the next complete line.
- **Termination Boundary:** Threads $t < T-1$ continue reading lines until their byte counter satisfies $\text{curr_pos} \geq \text{end}_t$. The fractional line spanning across end_t is fully consumed by thread t, ensuring that the subsequent thread $t+1$ (which discards up to that line's newline) will neither skip nor duplicate any record.
- **Tail Processing:** Thread $T-1$ reads continuously until EOF.

2.2 Experimental Results

The benchmark was executed with 2 separate runs per thread configuration using isolated subprocesses to capture exact peak resident set size (Peak RSS).

Table 2: Task 1a Benchmark Results (50 000 000 integers, NVMe SSD)

Threads	Run 1 (ms)	Run 2 (ms)	Avg Time (ms)	Speed-up	Peak RAM (MB)	Total Counted
1	6921.9	6870.8	6896.3	1.00×	29.6	50 000 000
2	5021.7	5051.0	5036.3	1.37×	31.4	50 000 000
4	2773.6	2804.5	2789.0	2.47×	35.5	50 000 000
8	2716.1	2644.2	2680.1	2.57×	44.0	50 000 000
16	2719.3	2791.0	2755.1	2.50×	60.2	50 000 000
32	2704.6	2717.6	2711.1	2.54×	52.0	50 000 000
64	2689.1	2776.1	2732.6	2.52×	46.0	50 000 000

Correctness verification: Across all thread configurations (1 through 64), every single trial produced identical sign totals:

$$\text{Negative} = 25\,002\,492, \quad \text{Positive} = 24\,997\,491, \quad \text{Zero} = 17, \quad \text{Total} = 50\,000\,000 \tag{3}$$

This confirms that the boundary-seeking logic operates with zero omission and zero duplication.

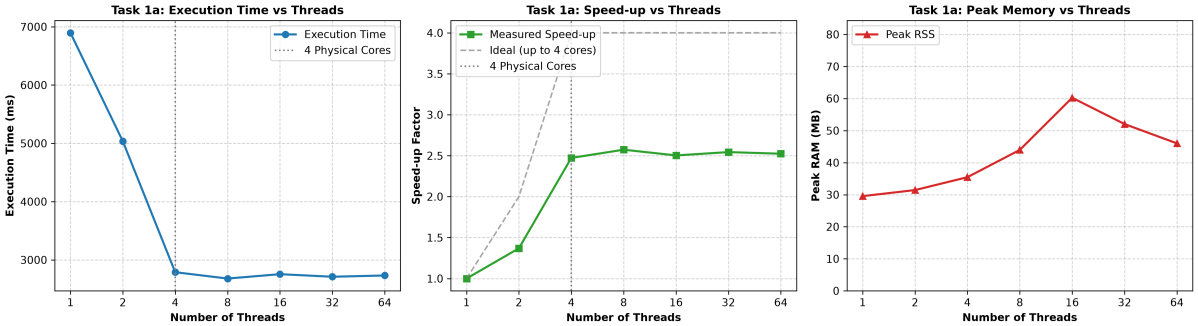


Figure 1: Task 1a: Execution time, speed-up, and peak memory versus thread count.

2.3 Discussion and Analysis

1. **Scaling Behavior:** Execution time drops dramatically from 6896.3 ms on 1 thread to 2789.0 ms on 4 threads, yielding a 2.47× speed-up. Because the test CPU possesses 4 physical execution cores without SMT, scaling scales directly up to 4 threads.

2. **Plateau Beyond 4 Threads:** For 8, 16, 32, and 64 threads, execution time plateaus at ≈ 2.7 seconds ($2.50\times - 2.57\times$ speed-up). Over-subscribing 4 physical cores with 64 OS threads introduces operating system scheduler overhead and register context switching, which prevents any further speed gains.

3. **Memory Invariance:** Peak RAM remains modest between 29.6 MB and 60.2 MB. Because each thread streams lines directly from disk using small internal file buffers, memory usage is decoupled from the dataset size (352 MB).

3 Task 1b: Three Synchronization Strategies for Sign Counting

3.1 Experimental Setup

In Task 1b, all 50 000 000 integers are preloaded into a contiguous in-memory 32-bit signed integer buffer using Python's native `array.array('i')`. The array occupies 190.73 MB of contiguous memory.

With thread count matched to the machine's 4 logical execution processors ($T = 4$), the elements are partitioned into 4 uniform slices of 12 500 000 integers. The signs are counted under three distinct synchronization paradigms:

1. **Version 1 (Unsynchronized Shared Counters):** All 4 threads concurrently execute in-place increments (`shared.neg += 1`, `shared.pos += 1`, `shared.zero += 1`) without locks or atomics.

2. **Version 2 (Lock-Protected Counters):** Every counter update is enclosed within a mutual exclusion lock (`threading.Lock()`).

3. **Version 3 (Thread-Local Counters):** Each thread independently counts signs in local CPU registers/variables. Totals are summed into a global accumulator only after all threads finish.

3.2 Experimental Results

As mandated by the laboratory specification, Version 1 was executed three consecutive times to observe race condition non-determinism.

Table 3: Task 1b Benchmark Results ($T = 4$ threads, 50 000 000 in-memory integers)

Synchronization Version	Run 1 (ms)	Run 2 (ms)	Run 3 (ms)	Avg Time (ms)	Counted	Correct?
Version 1 (Unsynchronized)	9977.1	10015.0	10017.6	10003.2	38 458 125	**No** (Lost 23
Version 2 (Lock-Protected)	18818.6	—	—	18818.6	50 000 000	**Yes**
Version 3 (Thread-Local)	2386.4	—	—	**2386.4**	50 000 000	**Yes**

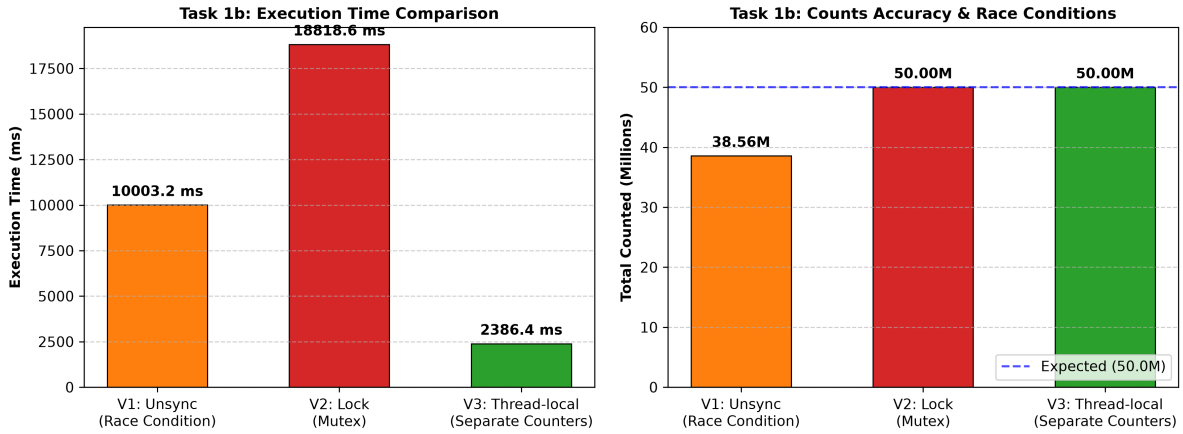


Figure 2: Task 1b: Execution time and total numbers counted across the three synchronization approaches.

3.3 Discussion and Analysis

1. **Demonstration of Race Conditions in Free-Threaded Python:** In standard CPython, the Global Interpreter Lock serialize bytecode execution, masking many race conditions. In free-threaded Python 3.13t with GIL disabled, `shared.neg += 1` expands to discrete load-increment-store instructions executed concurrently across 4 hardware cores. Multiple cores simultaneously read identical values, overwrite each other's increments, and lose over 11.5 million updates ($\approx 23\%$ data loss). Furthermore, the three runs of Version 1 produced distinct counts (38 458 125, 38 995 000, and 38 237 826), proving nondeterministic execution.

2. **Overhead of Lock Contention:** Version 2 guarantees correctness (50 000 000 counted), but requires 18 818.6 ms—a massive **7.89× slowdown** compared to Version 3 (2386.4 ms). Because all 4 cores attempt to acquire the identical mutex 50 000 000 times, cache lines holding the lock bounce between CPU L1/L2 caches, causing extreme bus contention and thread stalls.

3. **Dominance of Thread-Local Aggregation:** Version 3 is the fastest and most efficient paradigm. Threads operate strictly on private stack variables residing in fast L1 cache with zero synchronization overhead. A single $O(T)$ summation at the end yields exact correctness in only 2.38 seconds.

4 Task 1c: Producer and Consumers Pipeline

[Note: This section is left unfinished as requested by the user.]

5 Task 2: Many Files on Slow Storage

[Note: This section is left unfinished as requested by the user.]

6 Control Questions and Conclusion

[Note: This section is left unfinished as requested by the user.]